LAPORAN PRAKTIKUM STRUKTUR DATA

PEKAN 4

“QUEUE”

OLEH:

LEXI MULIA YUNASPI

2511531006

MATA KULIAH STRUKTUR DATA
DOSEN PENGAMPU : DR WAHYUDI,MT

ASISTEN PRAKTIKUM : M.GHALID

FAKULTAS TEKNOLOGI INFORMASI
DEPARTEMEN INFORMATIKA
UNIVERSITAS ANDALAS

2026

KATA PENGANTAR

Puji syukur penulis ucapkan kehadirat Allah SWT karena atas rahmat dan karunia-Nya laporan praktikum Struktur Data Pekan 4 tentang Queue ini dapat diselesaikan dengan baik. Laporan ini dibuat sebagai salah satu bentuk dokumentasi dan pemahaman penulis terhadap materi queue menggunakan array, linked list, iterasi queue, reverse queue, serta penerapan queue pada simulasi antrian loket menggunakan bahasa pemrograman Java.

Penulis mengucapkan terima kasih kepada dosen pengampu mata kuliah Struktur Data, asisten praktikum, serta semua pihak yang telah membantu selama proses praktikum berlangsung. Dengan adanya praktikum ini, penulis dapat memahami konsep queue serta penerapannya dalam pemrograman secara lebih baik.

Penulis menyadari bahwa laporan ini masih memiliki kekurangan. Oleh karena itu, penulis mengharapkan kritik dan saran yang membangun agar laporan selanjutnya dapat menjadi lebih baik. Semoga laporan praktikum ini dapat bermanfaat bagi pembaca dan menjadi tambahan wawasan dalam mempelajari struktur data queue.

Padang, 2026

Penulis

DAFTAR ISI

KATA PENGANTAR…….……..……..…....i

DAFTAR ISI…….……..…..…..…ii

BAB I PENDAHULUAN……..…….....…1

1.1 Latar Belakang……..……....…1

1.2 Manfaat……..……...…....1

1.3 Tujuan…….….…..1

BAB II PEMBAHASAN……..……..…2

2.1 Langkah Langkah Praktikum……..…..…..……..2

2.2 Langkah Langkah Tugas Praktikum…….....…….…..13

BAB III PENUTUP……..……....…...20

3.1 Kesimpulan…….…….…..…..…20

3.2 Saran……..……..……...….20

DAFTAR PUSTAKA……..……..….……..…21

BAB I

PENDAHULUAN

1.1 Latar Belakang

Struktur data merupakan cara untuk menyimpan, mengatur, dan mengelola data agar dapat digunakan secara efisien dalam pemrograman. Salah satu struktur data yang sering digunakan adalah queue atau antrian. Queue bekerja dengan konsep FIFO (First In First Out), yaitu data yang pertama masuk akan menjadi data pertama yang keluar. Konsep ini banyak diterapkan dalam kehidupan sehari-hari, seperti antrian loket, antrian kendaraan, dan sistem pelayanan lainnya.Dalam praktikum pekan 4 ini dilakukan pembelajaran mengenai implementasi queue menggunakan array dan linked list pada bahasa pemrograman Java. Selain itu dipelajari juga operasi-operasi dasar queue seperti enqueue, dequeue, front, rear, display, iterasi queue, serta reverse queue menggunakan stack. Praktikum ini bertujuan agar mahasiswa memahami konsep queue dan mampu menerapkannya dalam pembuatan program sederhana.

1.2 Manfaat

1. Memahami konsep dasar struktur data queue.
2. Mengetahui cara implementasi queue menggunakan array dan linked list.
3. Memahami penggunaan method pada operasi queue seperti enqueue dan dequeue.
4. Melatih kemampuan pemrograman Java dalam penerapan struktur data.
5. Menambah pemahaman mengenai penerapan queue pada simulasi antrian.

1.4 Tujuan

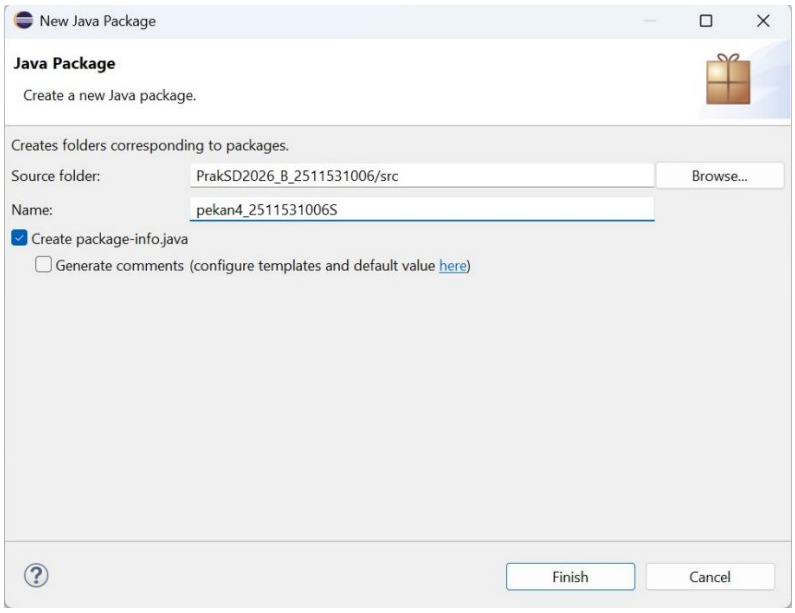
1. Mempelajari implementasi queue menggunakan bahasa pemrograman Java.
2. Memahami cara kerja queue dengan konsep FIFO.
3. Menggunakan method queue seperti enqueue, dequeue, front, rear, dan display.

BAB II

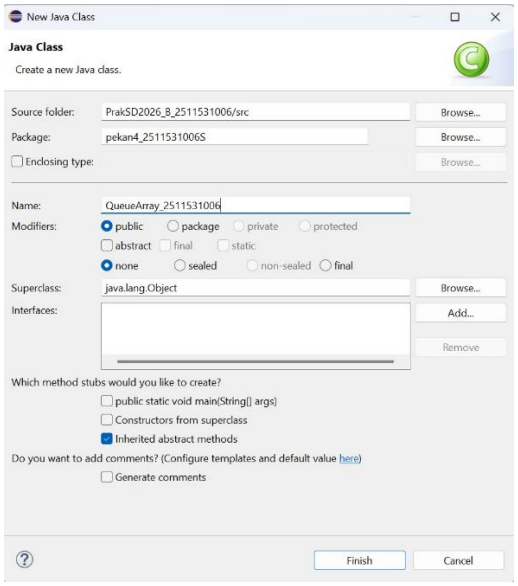
PEMBAHASAN

2.1 Langkah Langkah Praktikum

1. Program 1
   a. Pada tahap awal dilakukan pembuatan package baru di Eclipse dengan nama pekan4_2511531006S. Package ini digunakan untuk mengelompokkan file program agar lebih rapi dan mudah dikelola di dalam project Java.



   b. Selanjutnya dibuat class baru bernama QueueArray_2511531006. Class ini digunakan sebagai tempat penulisan program queue atau antrian menggunakan struktur data array.



c. Kemudian dituliskan deklarasi package dan class utama beserta variabel front, rear, size, capacity, dan array[]. Variabel tersebut digunakan untuk mengatur posisi antrian, kapasitas queue, jumlah data, serta penyimpanan elemen queue.

```
1  package pekan4_2511531006S;
2
3  public class QueueArray_2511531006<QueueArray> {
4      int front, rear, size;
5      int capacity;
6      int array[];
```

d. Berikutnya dibuat constructor QueueArray_2511531006(int capacity) yang berfungsi untuk menginisialisasi kapasitas queue, menentukan nilai awal front dan rear, serta membuat array sesuai ukuran yang diinginkan.

```
7
8      public QueueArray_2511531006(int capacity) {
9          this.capacity = capacity;
10         front = this.size = 0;
11         rear = capacity - 1 ;
12         array = new int[this.capacity];
13     }
14
```

e. Selanjutnya terdapat method isFull() yang digunakan untuk memeriksa apakah queue sudah penuh, sehingga data baru tidak bisa ditambahkan lagi.

```java
    boolean isFull(QueueArray_2511531006 queue) {
        return (queue.size == queue.capacity);
        }

    boolean isEmpty(QueueArray_2511531006 queue) {
        return (queue.size == 0);
    }

```

f. Selain itu terdapat method isEmpty() yang digunakan untuk mengecek apakah queue masih kosong atau belum memiliki elemen.

```java

    void enqueue_2511531006(int item) {
        if (isFull(this))
            return;
        this.rear = (this.rear + 1) % this.capacity;
        this.array[this.rear] = item;
        this.size = this.size + 1;
        System.out.println(item +" enqueued to queue");
    }

```

g. Program juga memiliki method enqueue_2511531006(int item) yang berfungsi untuk menambahkan data baru ke dalam queue. Setelah data masuk, posisi rear dan ukuran queue akan diperbarui.

```java
    int dequeue_2511531006() {
        if (isEmpty(this))
            return Integer.MIN_VALUE;
        int item = this.array[this.front];
        this.front = (this.front + 1) % this.capacity;
        this.size = this.size - 1;
        return item;
    }
```

h. Kemudian dibuat method dequeue_2511531006() yang digunakan untuk menghapus data paling depan dari queue. Setelah proses penghapusan, posisi front akan bergeser dan jumlah data berkurang.

```java
    int front_2511531006() {
        if (isEmpty(this))
            return Integer.MIN_VALUE;

      return this.array[this.front];
    }
```

i. Selanjutnya terdapat method front_2511531006() yang digunakan untuk melihat elemen paling depan pada queue tanpa menghapus data tersebut.

```
    int rear_2511531006() {
        if (isEmpty(this))
            return Integer.MIN_VALUE;
        return this.array[this.rear];
    }
```

j. Berikutnya dibuat method rear_2511531006() yang berfungsi untuk menampilkan elemen paling belakang dari queue.

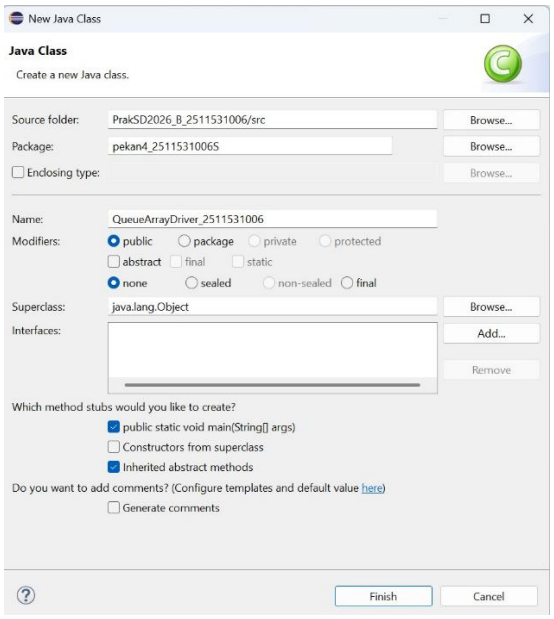
```
    //mencetak elemen antrian
    void display_2511531006() {
        int i;
        if (front == rear) {
            System.out.print("\nAntrian Kosong\n");
            return;
    }
```

k. Pada bagian akhir dibuat method display_2511531006() untuk menampilkan seluruh isi queue. Program akan mengecek apakah queue kosong, lalu menggunakan perulangan for untuk mencetak semua elemen antrian dari depan hingga belakang.

```
    //Kunjungi dari belakang dan cetak
        for (i = front; i < rear; i++) {
            System.out.printf(" %d <--", array[i]);
    }
        return;
    }

}
```

2. Program 2

a. Pada tahap awal dibuat class baru bernama QueueArrayDriver_2511531006 di dalam package pekan4_2511531006S. Pada saat pembuatan class, opsi public static void main(String[] args) dipilih agar program dapat langsung dijalankan.



b. Selanjutnya dituliskan deklarasi package dan class utama QueueArrayDriver_2511531006. Di dalam class tersebut terdapat method main() yang menjadi titik awal eksekusi program Java.

```
1  package pekan4_2511531006S;
2
3  public class QueueArrayDriver_2511531006 {
4
5      public static void main(String[] args) {
```

c. Kemudian dibuat objek queue dengan nama queue menggunakan class QueueArray_2511531006 dengan kapasitas 1000. Setelah itu program menambahkan beberapa data ke dalam queue menggunakan method enqueue_2511531006(), yaitu data 10, 20, 30, dan 40.

```
6          QueueArray_2511531006 queue = new QueueArray_2511531006(1000)
7          queue.enqueue_2511531006(10);
8          queue.enqueue_2511531006(20);
9          queue.enqueue_2511531006(30);
10         queue.enqueue_2511531006(40);
```

d. Berikutnya program menampilkan elemen paling depan dan paling belakang queue menggunakan method front_2511531006(). Setelah itu program juga menampilkan seluruh isi queue menggunakan method display_2511531006().

```java
        System.out.println("Item di depan "+ queue.front_2511531006());
        System.out.println("Item di belakang "+ queue.front_2511531006());
        System.out.println("tampilan queue");
        queue.display_2511531006();
```

e. Selanjutnya program melakukan penghapusan satu data dari queue menggunakan method dequeue_2511531006(). Setelah data dihapus, program kembali menampilkan elemen paling depan, elemen paling belakang, dan isi queue terbaru setelah penghapusan data dilakukan.

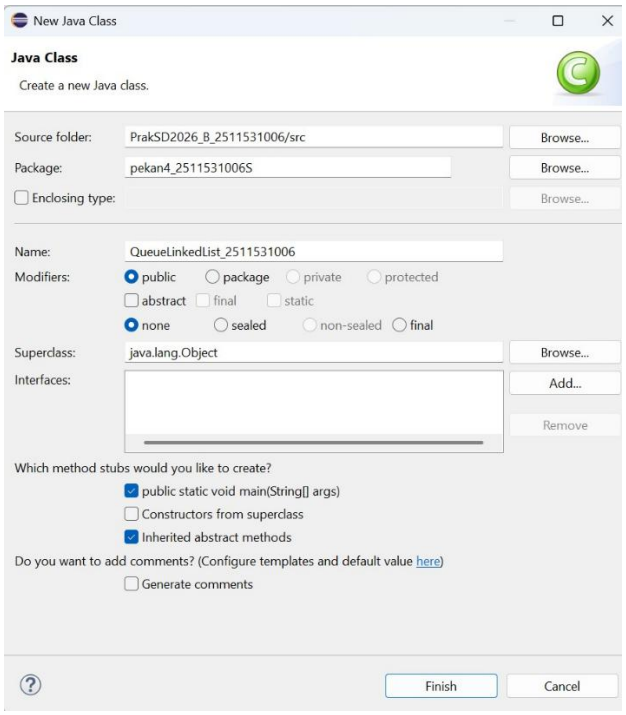
```java
        System.out.println(queue.dequeue_2511531006() +" dihapus dari queue");
        System.out.println("item di depan:" + queue.front_2511531006());
        System.out.println("item di belakang: " + queue.front_2511531006());
        System.out.println("tampilan queue setelah satu data dihapus");
        queue.display_2511531006();

    }

}
```

f. Pada bagian akhir terlihat hasil output program pada console. Output menunjukkan bahwa data berhasil ditambahkan ke queue, data paling depan dan belakang berhasil ditampilkan, serta queue berhasil diperbarui setelah satu elemen dihapus dari antrian.

```
Console
<terminated> QueueArrayDriver_2511531006 [Java Application] C
10 enqueued to queue
20 enqueued to queue
30 enqueued to queue
40 enqueued to queue
Item di depan 10
Item di belakang 10
tampilan queue
 10 <-- 20 <-- 30 <--10 dihapus dari queue
item di depan:20
item di belakang: 20
tampilan queue setelah satu data dihapus
 20 <-- 30 <--
```

3. Program 3

a. Pada tahap awal dibuat class baru bernama QueueLinkedList_2511531006 di dalam package pekan4_2511531006S. Pada pembuatan class dipilih method public static void main(String[] args) agar program dapat langsung dijalankan.



b. Selanjutnya program mendeklarasikan package dan melakukan import java.util.LinkedList. Import ini digunakan agar class LinkedList dapat dipakai untuk membuat struktur data queue berbasis linked list.

```
1  package pekan4_2511531006S;
2
3⊕ import java.util.LinkedList;
5
6  public class QueueLinkedList_2511531006 {
7
8
9⊖     public static void main(String[] args) {
```

c. Kemudian dibuat objek queue dengan nama q_2511531006 menggunakan LinkedList<Integer>. Setelah itu program menggunakan perulangan for untuk menambahkan elemen 0 sampai 5 ke dalam antrian menggunakan method add().

```
10          Queue<Integer> q_2511531006 = new LinkedList<>();
11          // tambah elemen {0, 1, 2, 3, 4,5} ke anterian
12          for (int i = 0; i < 6; i++)
13              q_2511531006.add(i);
```

d. Berikutnya program menampilkan seluruh isi queue menggunakan System.out.println(), sehingga semua elemen yang telah dimasukkan ke antrian dapat terlihat pada output.

```
        // menampilkan isi antrian.
        System.out.println("Elemen Antrian " + q_2511531006);
```

e. Selanjutnya program menggunakan method remove() untuk menghapus elemen paling depan dari queue. Data yang dihapus disimpan ke variabel hapus, kemudian hasil penghapusan dan isi queue terbaru ditampilkan ke layar.

```
        // Untuk menghapus kepala antrian.
        int hapus = q_2511531006.remove();
        System.out.println("Hapus elemen = " + hapus);
        System.out.println(q_2511531006 );
```

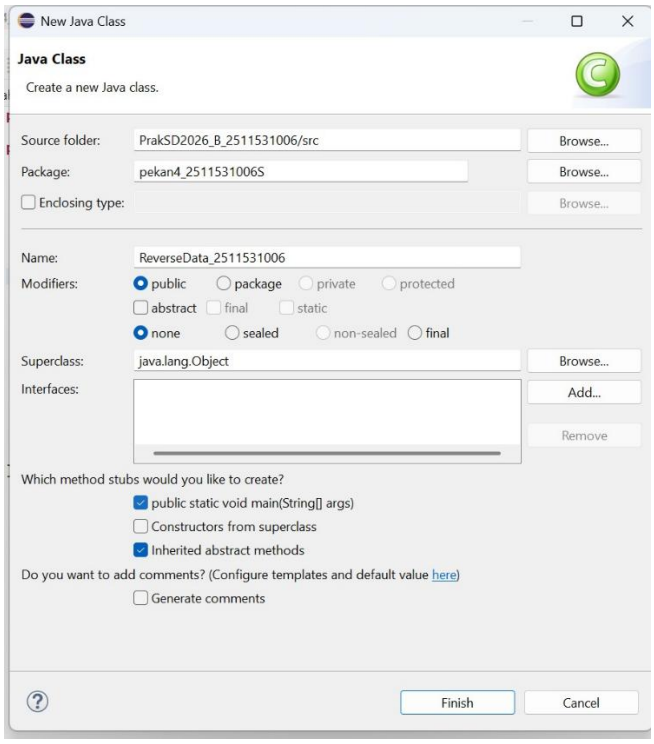
f. Pada bagian akhir program menggunakan method peek() untuk melihat elemen paling depan tanpa menghapusnya dari antrian. Setelah itu method size() digunakan untuk menghitung jumlah elemen yang masih ada di dalam queue dan menampilkannya ke output program.

```
        // Untuk melihat antrian terdepan
        int depan = q_2511531006.peek();
        System.out.println("Kepalan antrian " + depan);

        int banyak = q_2511531006.size();
        System.out.println("Size antrian = " + banyak);


    }

}
```

g. Output pertama menampilkan isi queue [0, 1, 2, 3, 4, 5] yang ditambahkan menggunakan method add().Selanjutnya method remove() menghapus elemen terdepan yaitu 0, sehingga isi queue berubah menjadi [1, 2, 3, 4, 5].Method peek() menampilkan kepala antrian 1, sedangkan size() menunjukkan jumlah elemen queue yang tersisa yaitu 5.

```
Console
<terminated> QueueLinkedList_2511531006 [Ja
Elemen Antrian [0, 1, 2, 3, 4, 5]
Hapus elemen = 0
[1, 2, 3, 4, 5]
Kepalan antrian 1
Size antrian = 5
```

4. Program 4
   a. Pada awal program dibuat class ReverseData_2511531006 di dalam package pekan4_2511531006S.Class ini digunakan untuk membuat program pembalikan data queue dengan menggunakan stack.Method main() dipilih agar program dapat langsung dijalankan.



b. Selanjutnya program mengimpor LinkedList, Queue, dan Stack dari package java.util.Class Queue dan LinkedList digunakan untuk membuat antrian data. Sedangkan class Stack digunakan untuk membantu proses membalik urutan data queue.

```
1  package pekan4_2511531006S;
2
3  import java.util.LinkedList;
4  import java.util.Queue;
5  import java.util.Stack;
6
7  public class ReverseData_2511531006 {
8
9
10
11     public static void main(String[] args) {
```

c. Kemudian dibuat objek queue q_2511531006 menggunakan LinkedList<Integer>. Method add() digunakan untuk menambahkan data 1, 2, dan 3 ke dalam queue. Setelah itu method println() menampilkan isi queue sebelum dilakukan reverse yaitu [1, 2, 3].

```
12             Queue<Integer> q_2511531006 = new LinkedList<Integer>();
13             q_2511531006.add(1);
14             q_2511531006.add(2);
15             q_2511531006.add(3); // [1, 2, 3]
16             System.out.println("sebelum reverse" + q_2511531006);
```

d. Berikutnya dibuat objek stack s menggunakan Stack<Integer>. Method isEmpty() digunakan untuk mengecek apakah queue masih memiliki data. Method remove() mengambil data dari queue lalu push() memasukkan data tersebut ke stack.

```
17             Stack<Integer> s = new Stack<Integer>();
18             while (!q_2511531006.isEmpty()) { // Q -> S
19                 s.push(q_2511531006.remove());
20             }
```

e. Selanjutnya program menggunakan perulangan kedua untuk memindahkan data dari stack ke queue.Method pop() mengambil data dari stack dan method add()

memasukkannya kembali ke queue.Proses ini membuat urutan data queue menjadi terbalik dari sebelumnya.

```
            while (!s.isEmpty()) { // S -> Q
                q_2511531006.add(s.pop());
            }
```

f. Pada bagian akhir method println() digunakan untuk menampilkan hasil reverse queue.

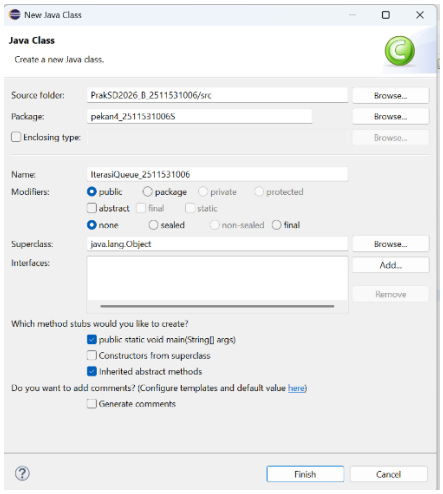
```
            System.out.println("sesudah reverse= " + q_2511531006); // [3, 2, 1]

    }

}
```

g. Output awal queue adalah [1, 2, 3], kemudian berubah menjadi [3, 2, 1]. Hasil tersebut menunjukkan bahwa proses pembalikan data queue berhasil dilakukan dengan bantuan stack.

```
Console
<terminated> ReverseData_2511531006 [Java Ap
sebelum reverse[1, 2, 3]
sesudah reverse= [3, 2, 1]
```

5. Program 5

a. Pada awal program dibuat class IterasiQueue_2511531006 di dalam package pekan4_2511531006S.



b. Method main() dipilih agar program dapat langsung dijalankan. Class ini digunakan untuk menampilkan isi queue menggunakan iterator.

```
1  package pekan4_2511531006S;
2
3  import java.util.Iterator;
4  import java.util.LinkedList;
5  import java.util.Queue;
6
7  public class IterasiQueue_2511531006 {
8
9    public static void main(String[] args) {
10       {
```

c. Selanjutnya program mengimpor Iterator, LinkedList, dan Queue dari java.util. Import ini digunakan untuk membuat queue dan melakukan proses iterasi data. Iterator berfungsi untuk mengakses elemen queue satu per satu.

```
11          Queue<String> q_2511531006 = new LinkedList<>();
12
```

d. Kemudian dibuat objek queue q_2511531006 menggunakan LinkedList<String>. Method add() digunakan untuk menambahkan data "Praktikum", "Struktur", "Data", "Dan", dan "Algoritma".Setelah itu dibuat objek iterator menggunakan method

iterator().

```
13            q_2511531006.add("Praktikum");
14            q_2511531006.add("Struktur");
15            q_2511531006.add("Data");
16            q_2511531006.add("Dan");
17            q_2511531006.add("Algoritma");
18            Iterator<String> iterator = q_2511531006.iterator();
```

e. Berikutnya program menggunakan perulangan while dengan method hasNext(). Method hasNext() digunakan untuk mengecek apakah masih ada data berikutnya pada queue.Sedangkan method next() digunakan untuk mengambil dan menampilkan elemen queue satu per satu.

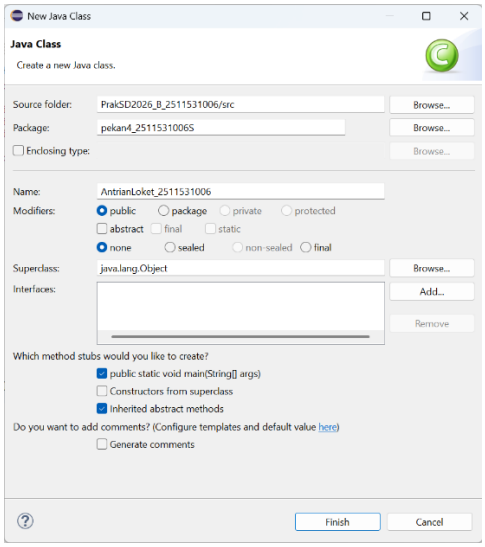
```
19            while (iterator.hasNext()) {
20                System.out.print(iterator.next() + " ");
21            }
22        }
23
24    }
25
26 }
```

f. Pada bagian akhir output program menampilkan isi queue yaitu Praktikum Struktur Data Dan Algoritma.Hasil tersebut menunjukkan bahwa iterator berhasil membaca seluruh elemen queue.Program berhasil menampilkan data queue secara berurutan.

```
Console
<terminated> IterasiQueue_2511531006 [Java Application
Praktikum Struktur Data Dan Algoritma
```

2.2 Langkah Langkah Tugas Praktikum

a. Pada awal program dibuat class AntrianLoket_2511531006 di dalam package pekan4_2511531006S. Class ini digunakan untuk membuat simulasi antrian loket berbasis array.Method main() dipilih agar program dapat langsung dijalankan.



b. Selanjutnya program mengimpor class Scanner dari java.util. Class Scanner digunakan untuk menerima input dari pengguna melalui keyboard. Input tersebut dipakai untuk memilih menu program dan memasukkan data pelanggan.

```
1  package pekan4_2511531006S;
2
3  import java.util.Scanner;
4
5  public class AntrianLoket_2511531006 {
6      private String[] queue;
7      private int front;
8      private int rear;
9      private int max;
```

c. Kemudian program mendeklarasikan variabel queue, front, rear, dan max.Array queue digunakan untuk menyimpan data antrian pelanggan.Sedangkan front dan rear digunakan untuk mengatur posisi depan dan belakang antrian.

```
10
11     public AntrianLoket_2511531006(int size) {
12         max = size;
13         queue = new String[max];
14         front = -1;
15         rear = -1;
16     }
17
```

d. Berikutnya dibuat constructor AntrianLoket_2511531006(int size).Constructor ini digunakan untuk menentukan ukuran maksimum queue dan membuat array antrian.Nilai awal front dan rear diatur menjadi -1 sebagai tanda antrian kosong.

```
    public void dequeue_2511531006() {
        if (isEmpty()) {
            System.out.println("Antrian kosong!");
            return;
        }
        System.out.println(queue[front] + " telah dilayani");
        for (int i = front; i < rear; i++) {
            queue[i] = queue[i + 1];
        }
        rear--;
        if (rear < 0) {
            front = rear = -1;
        }
    }

```

e. Method isEmpty() digunakan untuk mengecek apakah antrian kosong. Method ini akan mengembalikan nilai true jika front == -1. Pengecekan ini digunakan sebelum proses penghapusan atau penampilan data.

```
    public boolean isFull() {
        return rear == max - 1;
    }


```

f. Selanjutnya method isFull() digunakan untuk mengecek apakah antrian penuh. Method ini membandingkan nilai rear dengan kapasitas maksimum queue. Jika penuh maka data baru tidak dapat ditambahkan lagi.

```
    public void enqueue_2511531006(String nama) {
        if (isFull()) {
            System.out.println("Antrian penuh!");
            return;
        }
        if (front == -1) front = 0;
        queue[++rear] = nama;
        System.out.println("Data berhasil ditambahkan ke antrian");
    }

```

g. Method enqueue_2511531006() digunakan untuk menambahkan pelanggan ke dalam antrian.Program akan mengecek kondisi penuh menggunakan isFull(). Jika masih tersedia ruang, data pelanggan dimasukkan ke queue dan posisi rear bertambah.

```
    public void dequeue_2511531006() {
        if (isEmpty()) {
            System.out.println("Antrian kosong!");
            return;
        }
        System.out.println(queue[front] + " telah dilayani");
        for (int i = front; i < rear; i++) {
            queue[i] = queue[i + 1];
        }
        rear--;
        if (rear < 0) {
            front = rear = -1;
        }
    }

```

h. Berikutnya method dequeue_2511531006() digunakan untuk menghapus pelanggan terdepan.Program terlebih dahulu mengecek kondisi kosong menggunakan isEmpty(). Setelah data dihapus, elemen queue akan digeser dan posisi rear dikurangi.

```
    public void display_2511531006() {
        if (isEmpty()) {
            System.out.println("Antrian kosong!");
            return;
        }
        System.out.println("Isi antrian:");
        for (int i = front, nomor = 1; i <= rear; i++, nomor++) {
            System.out.println(nomor + ". " + queue[i]);
        }
    }

```

i. Method display_2511531006() digunakan untuk menampilkan seluruh isi antrian. Program menggunakan perulangan for untuk mencetak data pelanggan secara berurutan. Jika queue kosong maka program akan menampilkan pesan “Antrian kosong!”

```java
    public void Reverse_2511531006() {
        if (!isEmpty()) {
            for (int i = front, j = rear; i < j; i++, j--) {
                String temp = queue[i];
                queue[i] = queue[j];
                queue[j] = temp;
            }
        }
        display_2511531006();
    }

```

j. Selanjutnya method Reverse_2511531006() digunakan untuk membalik urutan antrian. Program menukar data depan dan belakang menggunakan variabel sementara temp. Hasil reverse kemudian ditampilkan menggunakan method display_2511531006().

```java

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        AntrianLoket_2511531006 antrian = new AntrianLoket_2511531006(10);

```

k. Pada method main(), program membuat objek Scanner dan objek antrian.Objek antrian dibuat dengan kapasitas maksimal 10 data pelanggan.Bagian ini menjadi awal menjalankan seluruh fitur program antrian loket.

```java
        System.out.println("=== PROGRAM ANTRIAN LOKET ===\n");
        System.out.println("1. Tambah Antrian");
        System.out.println("2. Hapus Antrian");
        System.out.println("3. Tampilkan Antrian");
        System.out.println("4. Reverse");
        System.out.println("5. Keluar\n");

```

l. Berikutnya program menampilkan menu pilihan menggunakan System.out.println(). Menu terdiri dari tambah antrian, hapus antrian, tampilkan antrian, reverse, dan keluar. Menu ini memudahkan pengguna memilih proses yang diinginkan.

```java
        int pilihan;
        do {
            System.out.print("Pilih menu: ");
            pilihan = sc.nextInt();
            sc.nextLine();
```

m. Selanjutnya program menggunakan perulangan do-while dan switch-case untuk menjalankan menu.Input pengguna dibaca menggunakan nextInt(), lalu setiap pilihan menjalankan method tertentu seperti enqueue, dequeue, display, dan Reverse.Program akan berhenti ketika pengguna memilih menu keluar.

```
            switch (pilihan) {
                case 1:
                    System.out.print("Masukkan nama pelanggan: ");
                    antrian.enqueue_2511531006(sc.nextLine());
                    break;
                case 2:
                    antrian.dequeue_2511531006();
                    break;
                case 3:
                    antrian.display_2511531006();
                    break;
                case 4:
                    antrian.Reverse_2511531006();
                    break;
                case 5:
                    System.out.println("Program selesai");
                    break;
                default:
                    System.out.println("Pilihan tidak valid!");
            }
        } while (pilihan != 5);

        sc.close();
    }
}
```

n. Pada output program pengguna memilih menu 1 untuk menambahkan pelanggan “Ana” dan “Bianca”.Data dimasukkan ke queue menggunakan method enqueue_2511531006(). Program menampilkan pesan bahwa data berhasil dimasukkan ke antrian.Terakhir pengguna mencoba menu tampilkan, reverse, hapus, dan keluar dari program. Method display_2511531006(), Reverse_2511531006(), dan dequeue_2511531006() berjalan dengan baik sesuai pilihan menu.Program berakhir setelah pengguna memilih menu 5 dan menampilkan pesan “Program selesai”.

Console

<terminated> AntrianLoket_2511531006 [Java Application] C:\Users

```
=== PROGRAM ANTRIAN LOKET ===

1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse
5. Keluar

Pilih menu: 1
Masukkan nama pelanggan: Ana
Data berhasil ditambahkan ke antrian
Pilih menu: 1
Masukkan nama pelanggan: Bianca
Data berhasil ditambahkan ke antrian
Pilih menu: 3
Isi antrian:
1. Ana
2. Bianca
Pilih menu: 4
Isi antrian:
1. Bianca
2. Ana
Pilih menu: 2
Bianca telah dilayani
Pilih menu: 3
Isi antrian:
1. Ana
Pilih menu: 5
Program selesai
```

BAB III

PENUTUP

3.1 Kesimpulan

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa struktur data queue bekerja dengan konsep FIFO (First In First Out), yaitu data yang pertama masuk akan menjadi data pertama yang keluar. Queue dapat diimplementasikan menggunakan array maupun linked list dengan method seperti enqueue, dequeue, front, rear, display, dan reverse.Pada praktikum ini juga dipelajari penggunaan iterator untuk menampilkan isi queue serta penggunaan stack untuk membalik urutan data queue. Selain itu, penerapan queue pada simulasi antrian loket menunjukkan bahwa queue sangat berguna dalam pengelolaan data secara teratur dan berurutan di dalam program Java.

3.2 Saran

Dalam melakukan praktikum struktur data, sebaiknya mahasiswa memahami terlebih dahulu konsep dasar queue sebelum membuat program agar lebih mudah memahami alur kerja method yang digunakan. Selain itu, ketelitian dalam penulisan syntax dan penggunaan method sangat diperlukan agar program dapat berjalan dengan baik tanpa error.Mahasiswa juga disarankan untuk lebih sering mencoba berbagai variasi program queue agar semakin memahami penerapan struktur data queue dalam kasus yang berbeda. Dengan latihan yang rutin, kemampuan pemrograman dan pemahaman logika program akan meningkat.

DAFTAR PUSTAKA

Data Structures and Algorithms in Java. 2014. *Data Structures and Algorithms in Java*. John Wiley & Sons.

Java: How to Program. 2018. *Java: How to Program*. Pearson Education